САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №2

по дисциплине «Методы оптимизации»

Тема: «Метод ломаных»

Выполнил:
Бен Шамех Абделазиз
Группа: К3339
Поток: МетОпт 1.2

Санкт-Петербург
2026 г.

Постановка задачи

Нужно написать программу, которая ищет глобальный минимум одномерной липшицевой функции на заданном отрезке методом ломаных (методом Пиявского). На вход подаются функция в виде строки, концы отрезка [a, b] и точность ε. Программа должна построить график функции, вспомогательных функций и итоговой ломаной, выдать приближённые значения аргумента и минимума, число итераций и затраченное время.

Работу нужно продемонстрировать на тестовых функциях хотя бы с двумя локальными минимумами на отрезке (Растригин и другие). Дополнительно к основной части выполнены два пункта: определение константы Липшица через вторую производную с объяснением принципа и подкаст-рефлексия.

1. Теоретические сведения

Функция W(x) называется липшицевой на [a, b], если найдётся такое L > 0, что для любых u, v из отрезка |W(u) − W(v)| ≤ L·|u − v|. Наименьшее такое L и есть константа Липшица; на практике подойдёт любая оценка сверху.

Идея метода в том, что минимум ищется не у самой функции, а у ломаной, которая всё время лежит под графиком. Для каждой уже вычисленной точки u_i строится «галочка»

$$g_i(x, u_i) = W(u_i) − L·|x − u_i|,$$

а ломаная порядка n — это максимум всех галочек:

$$p_n(x) = max_{i=0..n}\, g_i(x, u_i).$$

Из условия Липшица следует $p_n(x) \le W(x)$ на всём отрезке, то есть ломаная — миноранта. Верхние вершины ломаной лежат на графике W, нижние — между ними, под графиком.

Алгоритм. На стартовых итерациях берём $u_0 = a$ и $u_1 = b$ и считаем W в этих точках. Дальше на каждой итерации n находим $u_n = \operatorname{argmin} p_{n-1}(x)$, вычисляем $W(u_n)$, строим новую галочку и обновляем ломаную. Минимум ломаной достигается в одной из нижних вершин. Нижняя вершина между соседними верхними $u_q < u_v$ — это пересечение их галочек:

$$u_{cross} = (W(u_q) − W(u_v)) / (2L) + (u_q + u_v) / 2,$$
$$p(u_{cross}) = (W(u_q) + W(u_v)) / 2 − L·(u_v − u_q) / 2.$$

Поэтому на каждой итерации достаточно пройтись по соседним парам верхних вершин, выбрать пару с наименьшим p и взять её u_{cross} в качестве следующей точки.

Остановка: процесс заканчивается, когда $W(u_n) - p_{n-1}(u_n) < \varepsilon$ или исчерпан лимит итераций. Так как $p_{n-1}(u_n)$ — нижняя оценка глобального минимума, а лучшее найденное значение не больше $W(u_n)$, то оно отличается от истинного минимума меньше чем на ε. Эта нижняя оценка выводится программой отдельной строкой.

2. Константа Липшица через вторую производную

Для гладкой функции подходит $L = \max |W'(x)|$ на [a, b]. Чтобы найти этот максимум без перебора значений по мелкой сетке, используется вторая производная. Функция W' непрерывна на отрезке, поэтому её модуль достигает максимума либо на концах, либо во внутренней точке, где W' имеет экстремум, то есть где $W''(x) = 0$. Значит, достаточно проверить конечное число кандидатов: концы a, b и корни уравнения $W''(x) = 0$.

Порядок действий в программе: (1) sympy берёт символьные производные W' и W'' от введённой строки; (2) на отрезке ищутся промежутки, где W'' меняет знак (сетка нужна только для этого поиска), и корни в них уточняются методом Брента до 10^{-14}; (3) в каждом кандидате считается $|W'|$, берётся наибольшее значение, оно и есть L. Саму W' мы вычисляем точно, поэтому коэффициент запаса не нужен.

Пример: $W(x) = x + \sin(\pi x)$ на $[-2; 2]$. $W' = 1 + \pi\cdot\cos(\pi x)$, $W'' = -\pi^2\cdot\sin(\pi x)$. Нули W'' — точки $x = -1, 0, 1$, плюс концы ± 2. Значения $|W'|$: в $x = \pm 2$ и $x = 0$ получается $1 + \pi \approx 4{,}1416$, в $x = \pm 1$ получается $\pi - 1 \approx 2{,}1416$. Итак $L = 1 + \pi \approx 4{,}1416$.

Для сравнения в программе оставлен и численный способ: максимум $|\Delta W/\Delta x|$ по сетке из 5000 точек, умноженный на запас 1,2. Он не требует символьных производных, но даёт завышенное L, а чем больше L, тем более пологи нижние вершины и тем больше итераций нужно.

Таблица 1 – Сравнение двух способов оценки константы Липшица

Функция	**ε**	**L (2-я произв.)**	**L (сетка×1,2)**	**Итераций (2-я)**	**Итераций (сетка)**
Растригин	0,001	67,3327	80,7988	357	472
Из задания	0,01	4,1416	4,9699	11	14
Гриванк	0,001	1,0071	1,2085	208	280

Видно, что точная константа заметно сокращает число итераций (в среднем на четверть) при том же результате.

3. Программная реализация

Программа написана на Python (numpy, matplotlib, sympy, scipy) и состоит из нескольких функций. Тестовые функции хранятся в списке TESTS, новую можно добавить одной строкой.

g и p – галочка и ломаная

g строит галочку с наклоном ±L из точки u; значение W(u) передаётся готовым, чтобы функция не пересчитывалась. p – ломаная как максимум всех галочек, в основном цикле она не нужна, используется для графика.

```
def g(x, u, W_u, L):
    """ «галочка» g(x, u) = W(u) - L*|x - u| """
    return W_u - L * abs(x - u)

def p(x, u_list, W_list, L):
    """ ломаная p_n(x) = max_i g(x, u_i) """
    return max(g(x, u, w, L) for u, w in zip(u_list, W_list))
```

Рисунок 1 – Функции g и p

find_min_p – поиск нижней вершины

Для каждой пары соседних верхних вершин по формулам из теории считаются точка пересечения галочек и значение ломаной в ней. Выбирается пара с наименьшим значением. Значение ломаной берётся сразу по формуле, а не через максимум по всем галочкам, поэтому итерация стоит O(n).

```
def find_min_p(u_list, W_list, L):
    """
    Ищем u_n = argmin p_{n-1}(x).

    Минимум ломаной лежит в одной из нижних вершин, а нижняя вершина - это
    точка пересечения галочек двух соседних верхних вершин u_i и u_{i+1}:
        u_cross = (W_i - W_{i+1}) / (2L) + (u_i + u_{i+1}) / 2
        p(u_cross) = (W_i + W_{i+1}) / 2 - L*(u_{i+1} - u_i) / 2
    Значение в нижней вершине считаем сразу по формуле, поэтому за итерацию
    достаточно одного прохода по интервалам.
    """
    best_u, best_val = None, float("inf")

    for i in range(len(u_list) - 1):
        ui, uj = u_list[i], u_list[i + 1]
        Wi, Wj = W_list[i], W_list[i + 1]

        u_cross = (Wi - Wj) / (2 * L) + (ui + uj) / 2
        u_cross = min(max(u_cross, ui), uj)        # страховка от неточного L
        val = (Wi + Wj) / 2 - L * (uj - ui) / 2

        if val < best_val:
            best_u, best_val = u_cross, val

    return best_u, best_val
```

Рисунок 2 – Функция find_min_p

pyavsky – основной цикл

Стартуем с $u_0 = a$ и $u_1 = b$. На каждой итерации находим нижнюю вершину, вычисляем в ней W, вставляем новую точку в отсортированный список и проверяем условие $W(u_n) - p < \varepsilon$. В результат попадают лучшая точка, нижняя оценка минимума, число итераций и время.

```
def pyavsky(W, a, b, eps, L, max_iter=10000):
    """ метод ломаных на отрезке [a, b], останов по W(u_n) - p_{n-1}(u_n) < eps """
    t_start = time.perf_counter()

    # две стартовые верхние вершины: u_0 = a, u_1 = b
    u_list = [a, b]
    W_list = [W(a), W(b)]

    iterations = 0
    gap = float("inf")
    p_val = -float("inf")

    while iterations < max_iter:
        iterations += 1

        u_new, p_val = find_min_p(u_list, W_list, L)
        W_new = W(u_new)

        # новая верхняя вершина - вставляем с сохранением порядка по x
        idx = bisect.bisect_left(u_list, u_new)
        u_list.insert(idx, u_new)
        W_list.insert(idx, W_new)

        gap = W_new - p_val
        if gap < eps:
            break

    elapsed = time.perf_counter() - t_start
    best = int(np.argmin(W_list))

    return {
        "u_list": u_list,
        "W_list": W_list,
        "u_min": u_list[best],
        "W_min": W_list[best],
        "lower_bound": p_val,        # нижняя оценка глобального минимума
        "gap": gap,
        "iterations": iterations,
        "time": elapsed,
    }
```

Рисунок 3 – Функция pyavsky

lipschitz_second_derivative – L через вторую производную

Реализация принципа из раздела 2: символьные W' и W'', корни W'' на отрезке, максимум $|W'|$ среди корней и концов отрезка.

```python
def lipschitz_second_derivative(expr, a, b, n_scan=4000):
    """
    L = max |W'(x)| на [a, b].

    Принцип: на отрезке максимум |W'| достигается либо на концах, либо в точках,
    где W'' = 0 (там W' имеет экстремум). Поэтому
      1) берём W'(x) и W''(x) символьно (sympy);
      2) находим корни W''(x) = 0 на [a, b];
      3) считаем |W'| в этих корнях и на концах и берём наибольшее.

    Сетка n_scan нужна только чтобы найти промежутки, где W'' меняет знак;
    сами корни уточняются методом Брента, а L считается по точным значениям W'.
    """
    x = sp.Symbol("x")
    local = {"x": x, "pi": sp.pi, "e": sp.E}
    f = parse_expr(expr, local_dict=local)

    d1 = sp.diff(f, x)
    d2 = sp.diff(d1, x)
    f1 = sp.lambdify(x, d1, "numpy")
    f2 = sp.lambdify(x, d2, "numpy")

    xs = np.linspace(a, b, n_scan)
    v2 = np.broadcast_to(np.asarray(f2(xs), dtype=float), xs.shape)

    critical = [a, b]
    for i in range(len(xs) - 1):
        if v2[i] == 0.0:
            critical.append(xs[i])
        elif v2[i] * v2[i + 1] < 0:
            critical.append(brentq(lambda t: float(f2(t)), xs[i], xs[i + 1], xtol=1e-14))

    values = [abs(float(f1(c))) for c in critical]
    k = int(np.argmax(values))

    info = {
        "W'": str(d1),
        "W''": str(d2),
        "critical": sorted(set(critical)),
        "argmax": critical[k],
    }
    return values[k], info
```

Рисунок 4 – Оценка константы Липшица через вторую производную

lipschitz_grid и get_L – запасной способ

lipschitz_grid оценивает L по сетке с запасом 1,2. get_L выбирает способ и при ошибке разбора выражения автоматически переключается на сетку.

```python
def lipschitz_grid(W, a, b, n=5000, k=1.2):
    """ запасной вариант: max |ΔW/Δx| по сетке, умноженный на запас k """
    xs = np.linspace(a, b, n)
    ys = np.array([W(t) for t in xs])
    return float(np.max(np.abs(np.diff(ys) / np.diff(xs)))) * k

def get_L(expr, W, a, b, method="d2"):
    """ method = 'd2' - вторая производная, 'grid' - по сетке """
    if method == "d2":
        try:
            L, info = lipschitz_second_derivative(expr, a, b)
            return L, f"через 2-ю производную (max|W'| в x = {info['argmax']:.5f})"
        except Exception as err:
            print(f"не получилось через 2-ю производную ({err}), считаю по сетке")
    return lipschitz_grid(W, a, b), "по сетке с запасом 1.2"
```

Рисунок 5 – Оценка L по сетке и выбор способа

make_function и input_validation – функция из строки

Строка с выражением компилируется и вычисляется в ограниченном окружении, где доступны только математические функции. input_validation просит ввести выражение заново, пока оно не начнёт вычисляться.

```
def make_function(expr):
    """ строка с выражением -> функция W(x) """
    code = compile(expr, "<expr>", "eval")

    def W(x):
        env = dict(SAFE_ENV)
        env["x"] = x
        return eval(code, {"__builtins__": {}}, env)

    return W

def input_validation():
    """ спрашиваем выражение, пока оно не начнёт вычисляться """
    while True:
        expr = input("f(x) = ").strip()
        try:
            W = make_function(expr)
            W(0.0)
            return expr, W
        except Exception as err:
            print(f"ошибка в выражении: {err}, попробуйте ещё раз")
```

Рисунок 6 – Преобразование строки в функцию и проверка ввода

choose_function, ask_eps, ask_method, main – ввод данных

choose_function показывает меню тестовых функций или принимает свою, ask_eps и ask_method спрашивают точность и способ оценки L, main собирает всё вместе.

```
def choose_function():
    """ меню тестовых функций; возвращает (expr, W, a, b, name) """
    while True:
        print("\nвыберите тестовую функцию:")
        print("-" * 70)
        print(f"{'№':>2}  {'имя':<12} {'W(x)':<35} [a, b]")
        print("-" * 70)
        for i, t in enumerate(TESTS, 1):
            print(f"{i:>2}  {t['name']:<12} {t['expr']:<35} [{t['a']}, {t['b']}]")
        print(f"{len(TESTS) + 1:>2}  своя функция")
        print("-" * 70)

        choice = input("номер: ").strip()
        if not choice.isdigit():
            print("нужно ввести номер")
            continue

        n = int(choice)
        if 1 <= n <= len(TESTS):
            t = TESTS[n - 1]
            return t["expr"], make_function(t["expr"]), t["a"], t["b"], t["name"]

        if n == len(TESTS) + 1:
            expr, W = input_validation()
            try:
                a = float(input("a = "))
                b = float(input("b = "))
            except ValueError:
                print("a и b должны быть числами")
                continue
            if a >= b:
                print("должно быть a < b")
                continue
            return expr, W, a, b, "своя функция"

        print("такого номера нет")
```

Рисунок 7 – Меню выбора функции

```python
def ask_eps():
    while True:
        try:
            eps = float(input("eps = "))
            if eps > 0:
                return eps
            print("eps должно быть положительным")
        except ValueError:
            print("нужно ввести число")

def ask_method():
    print("\nкак искать константу Липшица?")
    print("  1 - через вторую производную (по умолчанию)")
    print("  2 - по сетке с запасом 1.2")
    return "grid" if input("номер: ").strip() == "2" else "d2"

def main():
    expr, W, a, b, name = choose_function()
    eps = ask_eps()
    method = ask_method()

    L, how = get_L(expr, W, a, b, method)
    print(f"\nфункция: {name}:  {expr}")
    print(f"отрезок: [{a}, {b}],  eps = {eps}")
    print(f"L = {L:.6f}  ({how})")

    result = pyavsky(W, a, b, eps, L)
    print_result(name, result, L)
    plot_result(W, a, b, result, L, title=f"{name}, eps = {eps}")
```

Рисунок 8 – Ввод точности, выбор способа оценки L и точка входа

print_result и plot_result – вывод

print_result печатает L, u*, W(u*), нижнюю оценку, число итераций и время. plot_result строит график: функцию, ломаную, галочки, верхние вершины и найденный минимум, а справа дополнительно увеличенную окрестность минимума.

```python
def print_result(name, result, L):
    print("\n" + "=" * 48)
    print(f"результаты ({name}):")
    print(f"  L                = {L:.6f}")
    print(f"  u*               = {result['u_min']:.10f}")
    print(f"  W(u*)            = {result['W_min']:.10f}")
    print(f"  нижняя оценка    = {result['lower_bound']:.10f}")
    print(f"  итераций         = {result['iterations']}")
    print(f"  время            = {result['time']:.4f} сек")
    print("=" * 48)
```

Рисунок 9 – Функция print_result

4. Результаты работы

Расчёты выполнены с константой Липшица, найденной через вторую производную. На графиках слева весь отрезок: синяя линия — W(x), красный пунктир — итоговая ломаная, тонкие зелёные линии — вспомогательные функции g_i, зелёные точки — верхние вершины, красная точка — найденный минимум. Справа — увеличенная окрестность найденного минимума.

Таблица 2 – Итоги расчётов

Функция	**[a, b]**	**ε**	**L**	**u***	**W(u*)**	**Нижняя оценка**	**Число итераций**	**Время, с**
Растригин	[−2; 3]	0,001	67,3327	1,000007	$8{,}6\cdot10^{-9}$	$-9{,}9\cdot10^{-4}$	357	0,045
Из задания	[−2; 2]	0,01	4,1416	−2,000000	−1,999995	−2,000000	11	0,0001
Гриванк	[−20; 20]	0,001	1,0071	0,000000	0	$-8{,}3\cdot10^{-4}$	208	0,016

4.1. Функция Растригина

$W(x) = 10 + (x - 1)^2 - 10\cdot\cos(2\pi x)$, отрезок [−2; 3], ε = 0,001. На отрезке несколько локальных минимумов, глобальный — в точке x = 1, где W = 0. Найдено u* = 1,000007, $W(u^*) = 8{,}6\cdot10^{-9}$. Видно, что точки сгущаются в окрестности глобального минимума, а около остальных локальных минимумов вычислений заметно меньше: ломаная там быстро поднимается выше текущего лучшего значения.

```
выберите тестовую функцию:
------------------------------------------------------------------------
 №  имя          W(x)                                    [a, b]
------------------------------------------------------------------------
 1  Растригин    10 + (x - 1)**2 - 10*cos(2*pi*x)        [-2.0, 3.0]
 2  Из задания   x + sin(3.14159*x)                      [-2.0, 2.0]
 3  Гриванк      1 + x**2/4000 - cos(x)                  [-20.0, 20.0]
 4  своя функция
------------------------------------------------------------------------
номер: 1
eps = 0.001

как искать константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
номер: 1

функция: Растригин:  10 + (x - 1)**2 - 10*cos(2*pi*x)
отрезок: [-2.0, 3.0],  eps = 0.001
L = 67.332659  (через 2-ю производную (max|W'| в x = -1.25081))

================================================
результаты (Растригин):
  L                = 67.332659
  u*               = 1.0000065907
  W(u*)            = 0.0000000086
  нижняя оценка    = -0.0009929417
  итераций         = 357
  время            = 0.0452 сек
================================================
```

Рисунок 10 – Запуск программы, функция Растригина

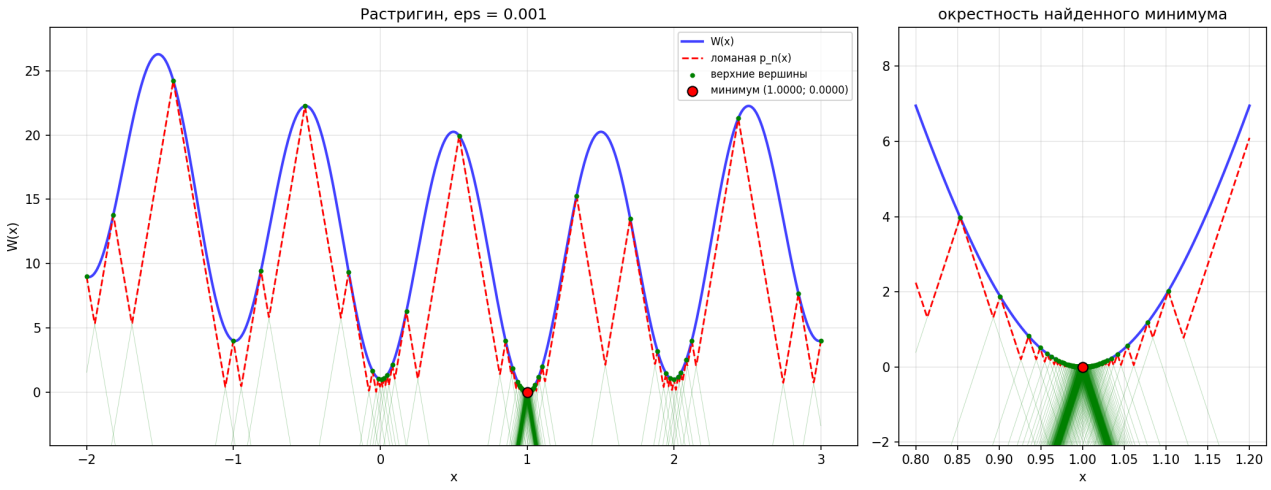


Рисунок 11 – График, функция Растригина, ε = 0,001

4.2. Функция из задания

W(x) = x + sin(3,14159·x), отрезок [−2; 2], ε = 0,01. Здесь два внутренних локальных минимума (x ≈ −0,60 и x ≈ 1,40), но глобальный минимум лежит на левом конце: u* = −2, W(u*) ≈ −2. Метод справился за 11 итераций.

```
выберите тестовую функцию:
------------------------------------------------------------------------
 №  имя          W(x)                                  [a, b]
------------------------------------------------------------------------
 1  Растригин    10 + (x - 1)**2 - 10*cos(2*pi*x)      [-2.0, 3.0]
 2  Из задания   x + sin(3.14159*x)                    [-2.0, 2.0]
 3  Гриванк      1 + x**2/4000 - cos(x)                [-20.0, 20.0]
 4  своя функция
------------------------------------------------------------------------
номер: 2
eps = 0.01

как искать константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
номер: 1

функция: Из задания:  x + sin(3.14159*x)
отрезок: [-2.0, 2.0],  eps = 0.01
L = 4.141590  (через 2-ю производную (max|W'| в x = 0.00000))

================================================
результаты (Из задания):
  L                = 4.141590
  u*               = -2.0000000000
  W(u*)            = -1.9999946928
  нижняя оценка    = -1.9999947487
  итераций         = 11
  время            = 0.0001 сек
================================================
```

Рисунок 12 – Запуск программы, функция из задания

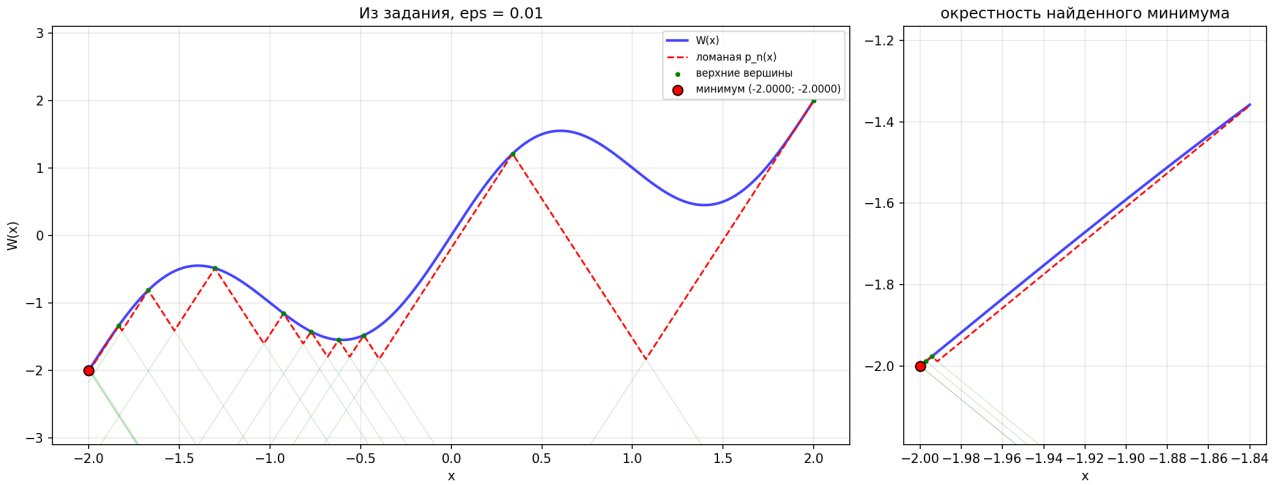


Рисунок 13 – График, функция из задания, ε = 0,01

4.3. Функция Гриванка (одномерная)

W(x) = 1 + x²/4000 − cos(x), отрезок [−20; 20], ε = 0,001. На отрезке около шести локальных минимумов с почти одинаковой глубиной, глобальный — в нуле. Найдено u* = 0, W(u*) = 0.

```
выберите тестовую функцию:
------------------------------------------------------------------------
 №  имя          W(x)                                    [a, b]
------------------------------------------------------------------------
 1  Растригин    10 + (x - 1)**2 - 10*cos(2*pi*x)    [-2.0, 3.0]
 2  Из задания   x + sin(3.14159*x)                  [-2.0, 2.0]
 3  Гриванк      1 + x**2/4000 - cos(x)              [-20.0, 20.0]
 4  своя функция
------------------------------------------------------------------------
номер: 3
eps = 0.001

как искать константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
номер: 1

функция: Гриванк:  1 + x**2/4000 - cos(x)
отрезок: [-20.0, 20.0],  eps = 0.001
L = 1.007069  (через 2-ю производную (max|W'| в x = -14.13767))

================================================
результаты (Гриванк):
  L                = 1.007069
  u*               = 0.0000000000
  W(u*)            = 0.0000000000
  нижняя оценка    = -0.0008325653
  итераций         = 208
  время            = 0.0156 сек
================================================
```

Рисунок 14 – Запуск программы, функция Гриванка

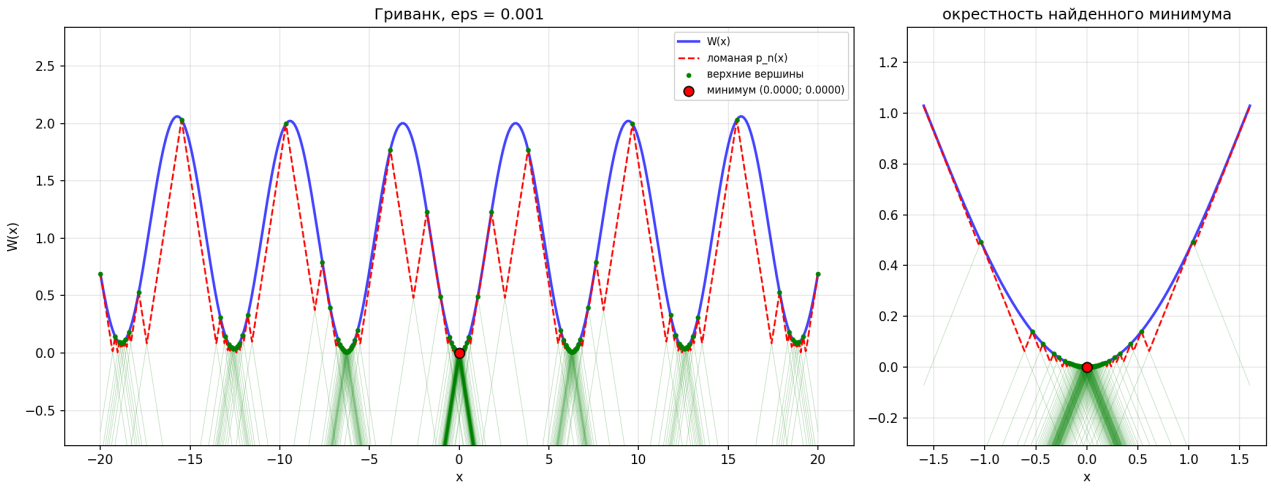


Рисунок 15 – График, функция Гриванка, ε = 0,001

Выводы

В работе реализован метод ломаных для поиска глобального минимума липшицевой функции и проверен на трёх функциях с несколькими локальными минимумами. Во всех случаях найден глобальный минимум с заданной точностью, а нижняя оценка $p_{n-1}(u_n)$ подтверждает, что отклонение от истинного значения не больше ε.

Сначала я не сразу понял, откуда берётся следующая точка: казалось, что нужно искать минимум ломаной по всему отрезку. После того как стало ясно, что он всегда в пересечении галочек соседних вершин, алгоритм сильно упростился, и значение ломаной в этой точке тоже считается по формуле, без перебора. Критерий остановки $W(u_n) - p_{n-1}(u_n) < \varepsilon$ оказался удобнее, чем просто сравнение соседних значений, потому что он сразу даёт гарантию точности.

Самым важным оказался выбор L. Оценка по сетке с запасом работает, но завышает константу и тратит лишние итерации (на тестах на 25–35% больше). Вычисление через вторую производную даёт точное значение: максимум $|W'|$ достигается на концах или в нулях W'', значит перебирать нужно всего несколько точек. Ограничение способа в том, что он требует дифференцируемой функции и символьного разбора выражения; для негладких функций приходится возвращаться к оценке по сетке, поэтому оба варианта оставлены в программе.

Код программы: [ссылка на репозиторий]